

Memoria Dinámica en C

Malak Sanchez

Monitorias Sistemas Operativos

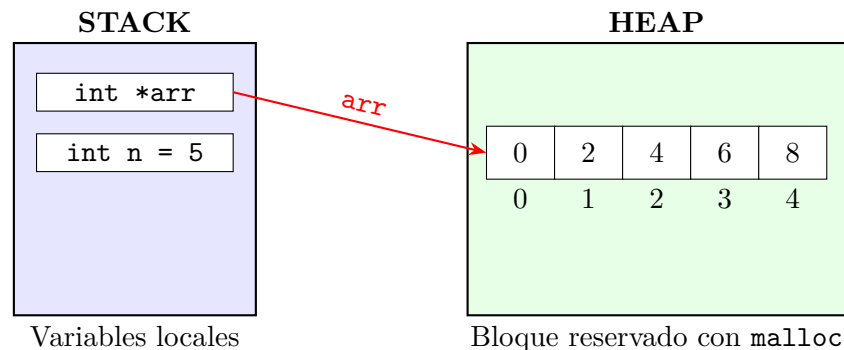
18 de septiembre de 2026

1. Motivación

En C, el tamaño de los arreglos normales debe conocerse antes de que el programa se ejecute. Sin embargo, en aplicaciones reales (como un sistema operativo gestionando procesos), no sabemos de antemano cuántos datos necesitaremos. La memoria dinámica nos permite solicitar espacio según sea necesario.

2. Idea Fundamental

La memoria dinámica permite reservar memoria durante la ejecución del programa (en el **Heap**), en lugar de definirla en tiempo de compilación (en el **Stack**).



3. Sintaxis Básica

Se utilizan funciones de la librería `<stdlib.h>`:

```
ptr = (tipo*) malloc(tamaño_en_bytes);
free(ptr);
```

4. Ejemplo Mínimo Funcional

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(){
5     int n = 5;
6     int *arr = (int*) malloc(n * sizeof(int));
7
8     if(arr == NULL) return 1; // Error al reservar
9
10    for(int i=0; i<n; i++) arr[i] = i * 2;
11
12    printf("Primer elemento: %d\n", arr[0]);
13
14    free(arr); // Importante: liberar la memoria
15    return 0;
16 }

```

5. Explicación Paso a Paso

1. Se calcula el tamaño necesario usando `n * sizeof(int)`.
2. `malloc` reserva ese espacio en el Heap y devuelve la dirección de inicio.
3. Se verifica si el puntero es `NULL` (falla en la reserva).
4. Se usa el puntero como si fuera un arreglo.
5. Se libera el espacio con `free` para que el sistema operativo pueda reutilizarlo.

6. Qué ocurre en memoria

Segmento	Variable	Estado
Stack	arr	Puntero (ej: 0x500)
Heap	[0x500...0x514]	Datos reservados (20 bytes)

7. Patrones comunes de uso

Creación de arreglos de tamaño variable:

```

1 int *crear_arreglo(int n){
2     return (int*) malloc(n * sizeof(int));
3 }

```

8. Errores Comunes

- **Memory Leak:** No usar `free()`, lo que agota la RAM del sistema.
- **Dangling Pointer:** Intentar acceder a la memoria después de haber hecho `free()`.

9. Buenas Prácticas

- Siempre verificar si `malloc` devolvió `NULL`.
- Asignar `NULL` al puntero inmediatamente después de liberar: `free(p); p = NULL;`.

10. Ejercicios

1. Pida al usuario un número N , cree un arreglo dinámico para N enteros, lea los valores y determine el menor valor almacenado.
2. Use `realloc` para reducir un arreglo dinámico de tamaño 6 a tamaño 3, conservando los primeros elementos.